

Gruppe 6

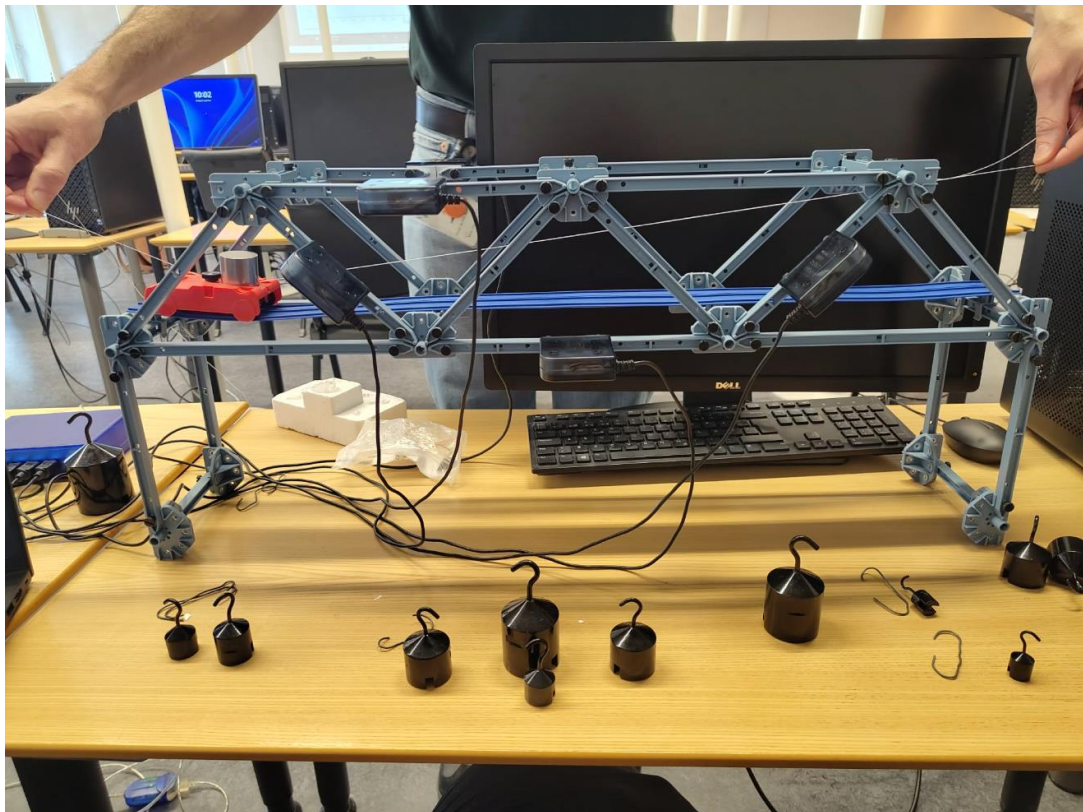
Dato: 20. september 2026

Medlemmer:

583411 - Lars Birger Bergmål

589829 - Madhi Karimi

674834 - Andrei Miguel Abaday Enriquez



HENSIKT

Hensikten med dette forsøket er å få en bedre intuisjon for hvordan kreftene fordeler seg i en bro, og hvordan kalkulasjonene ifra teorien stiller seg opp mot virkeligheten.

TEORI

Teorien er at ved å bruke klassisk mekanikk så skal en kunne estimere hvordan kreftene fordeler seg i konstruksjonen, og gjennom dette så skal en kunne bestemme hvor sterk konstruksjonen trenger å være for å ikke falle sammen under vanlig last.

HYPOTESE

Hypotesen er at de målte verdiene stemmer rundt omkring det vi har estimert, og at vi vil se at måleresultatene blir rundt omkring dobbelt så mye når vi flytter lodden til under ene fagverket.

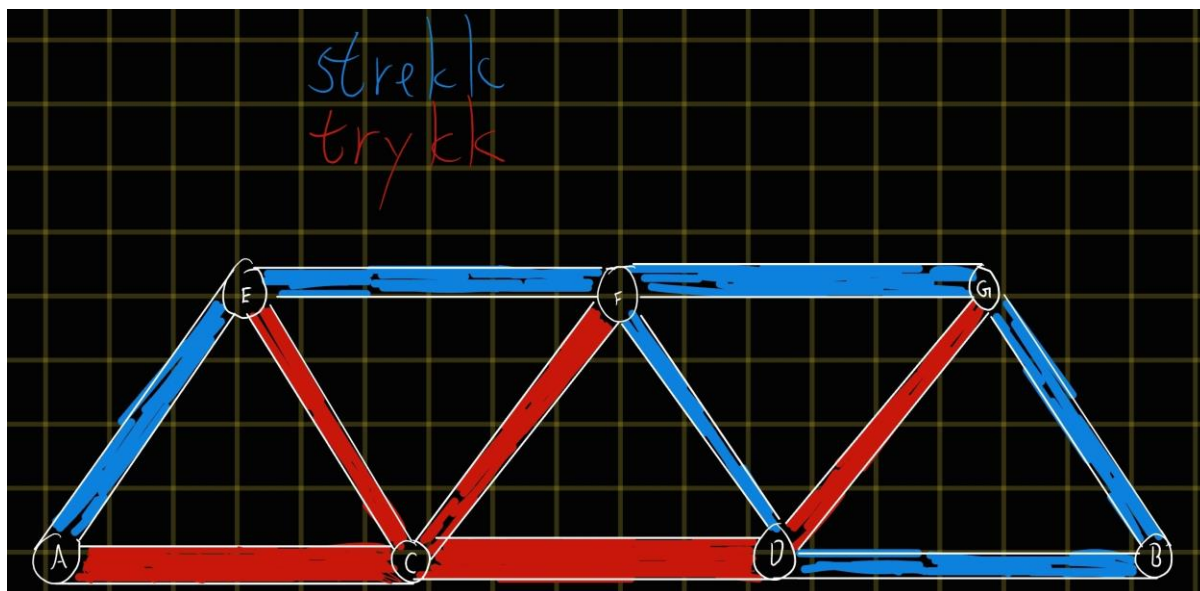
UTSTYR

-Datamaskin

- Pasco-Capstone programvare og lastcelleleser
- Vekter
- Fagverksbro med ferdigmonterte lastceller som kan kobles til lastcelleleseren
- Lekebil på 200g

Forberedende

Før lab forberedte vi oss med å regne ut kreftene i hver stav. Vi brukte et Python script for å finne svaret, og så puttet vi inn forskjellige verdier i scriptet for å regne ut for de forskjellige verdiene som hver person var gitt. Scriptet er linket til i Kilder, men er også skrevet i slutten av dokumentet. Grovt beskrevet så går det ut på å lage en likning for hver akse i hvert knutepunkt, og ha en variabel for hver stav, og løs det som et likningssett.



METODE

Dette gjorde vi på labben

1. a) Test 1:

- 1.1. Hang på lodd midt på de nedre tverrbjelkene, som tilsvarer den forhåndsberegnete lastesituasjonen.
- 1.2. Noter deg lastcellene. De fleste på den ene fagverksdelen og en på den andre, slik at det kan sjekkes om tilsvarende staver i hvert av de to fagverkene blir likt belastet.
- 1.3. Mål kreftene i de aktuelle stavene ved å trykke på startknappen i Capstone. Hvordan stemmer beregnede verdier og målte verdier overens?
- 1.4. Ta videre et skjermbilde av målingene i Capstone for senere bruk i labrapporten.
NB!!! Merk at data-programmet viser strekk-krefter med negativt fortegn og trykk-krefter med positivt fortegn, som er motsatt av fortegnsgreglene fra læreboka. Når du noterer måleresultatene inn i skjemaet på side 4, nytter du altså motsatt fortegn av det du leser av.

2. b) Test 2:

- 2.1. I dette forsøket skal du bare belaste den ene fagverksdelen ved å henge alle de 4 loddene i den oppgitte lastesituasjonen bare på den ene fagverkshalvdelen.
- 2.2. Noter resultatene dine inn i skjemaet på side 4.
- 2.3. Ta også et skjermbilde av målingene i Capstone for senere bruk i labrapporten.

3. c) Test 3:

- 3.1. I denne oppgaven skal du måle dynamisk belastning på fagverksbroen. Vi fester en vegbane til de horisontale bjelkene som forbinder de to fagverkshalvdelen.
- 3.2. Så lar vi en liten bil med last på 200 g kjøre over slik at den dynamiske belastningen på stavene blir registrert.
- 3.3. Ta også her et skjermbilde av målingene i Capstone for senere bruk i labrapporten.

NOTATER OG OBSERVASJONER

MAS149 Statikk og Fasthetslære Labb 1

Gruppe nr: 6

Navn studenter:

Student 1: Lars birger Bergmål

Student 2: Madhi Karimi

Student 3: Andrei Miguel Abaday Enriquez

Lastesituasjon =

Last 1 = 575[g], Last 2 = 425 [g], Last 3 = 250[g], Last 4 = 725[g]

Stavkrefter	Måleport nr.	Beregnet kraft (N)	Målt kraft (N)		Avvik (%)	
			Mellom fagverk	På et fagverk	Mellom fagverk	På et fagverk
A_x		- 0.0				
A_y		+ 2.04				
B		+ 13.32000				
N_{AC}		+ 3.59700				
N_{CD}	4	+ 6.62175	+ 6.06	+ 12.08	9.26	45.18
N_{BD}		- 3.02475				
N_{EF}	3	- 7.19400	- 7.50	- 13.30	4.08	45.90
N_{FG}		- 6.04950				
N_{AE}		- 5.08693				
N_{CE}	2	+ 5.08693	+ 5.02	+ 9.42	1.32	45.99
N_{CF}		+ 0.80928				
N_{DF}		- 0.80928				
N_{DG}	5	+ 4.27764	+ 3.83	+ 7.36	11.68	41.87
N_{BG}		- 4.27764				

Notater

Vi hadde dobbelt av vektene i beregningene plassert midt mellom fagverkene.

Det ble nevnt at dette var for å kompensere hvordan broen er bygget opp av 2 fagverk, og vekten vil bli fordelt dem imellom. (Test 1)

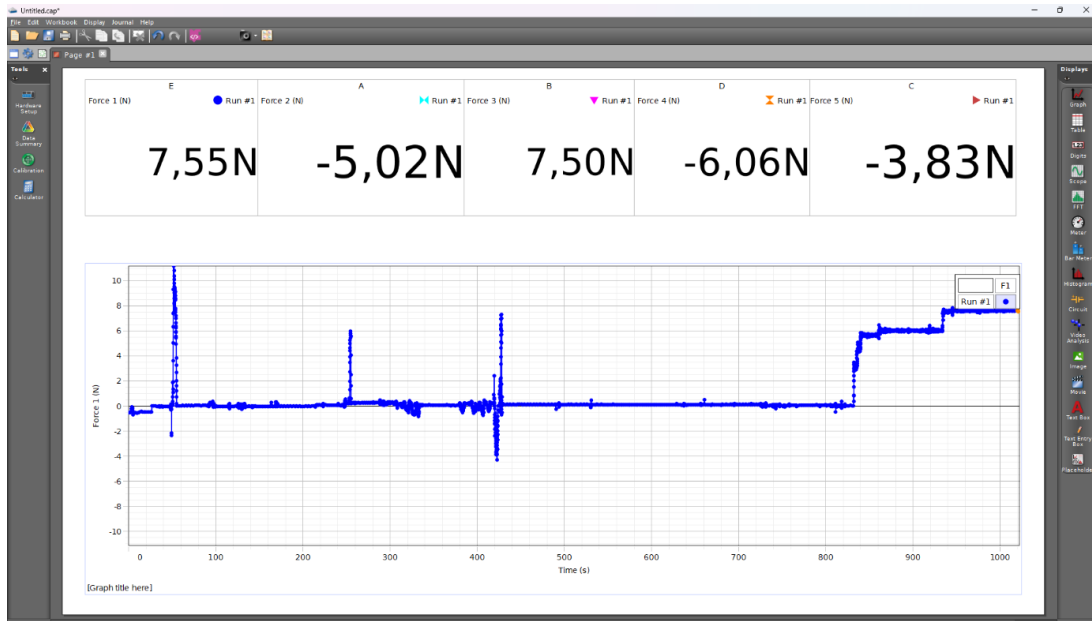
Vi merket i andre testen (Test 2), at de målte kreftene doblet seg når vi flyttet vektene fra midt mellom fagverkene til under ene fagverket.

Vi tror dette er for at kreftene fra vektene er annerledes fordelt når vektene er rett under ene fagverket.

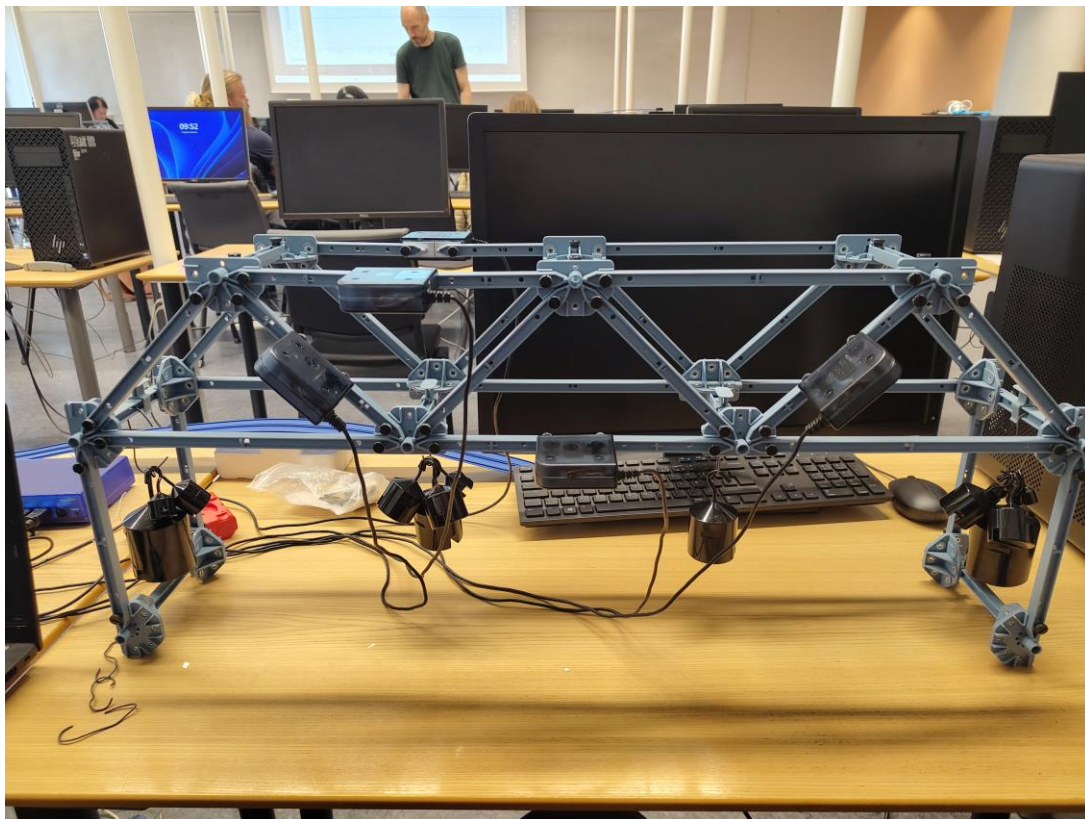
Under den dynamiske testen (Test 3), så ble et observert at alle de målte stavene hadde proporsjonale krefter, men noen som kompresjon og noen som strekk. Det ble nevnt at en trenger å simulere dette, der det ikke er noe linjær sak, sånn som i test 2 og 1.

Observasjoner

Test 1

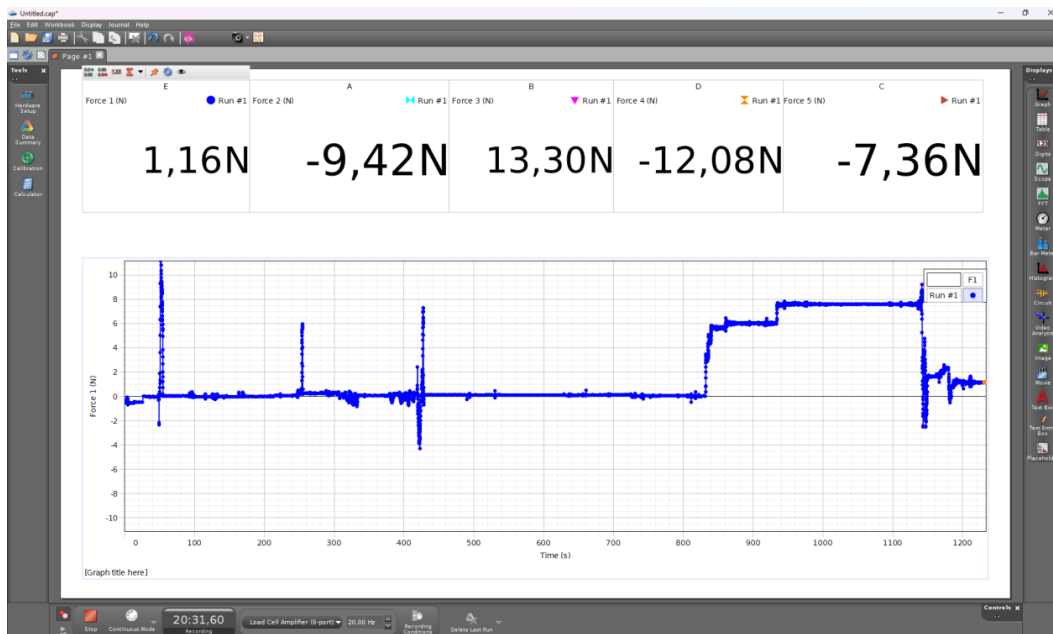


Måleresultat for test 1.

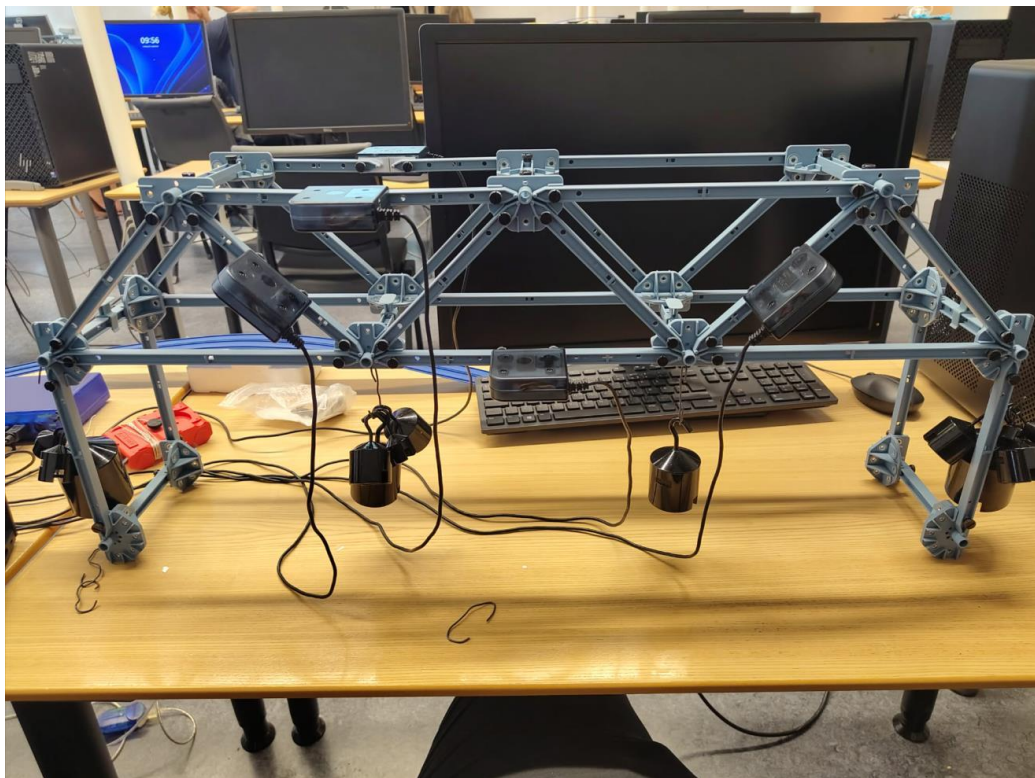


Bilde av broen under test 1

Test 2



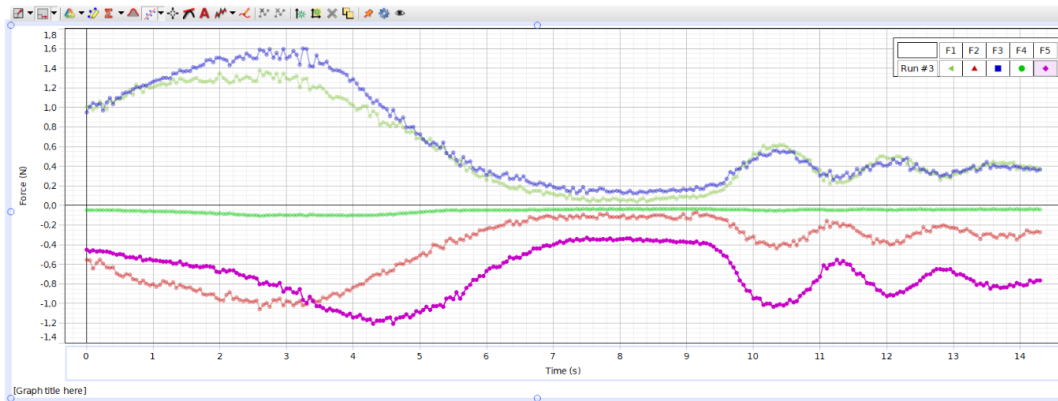
Måleresultat for test 2



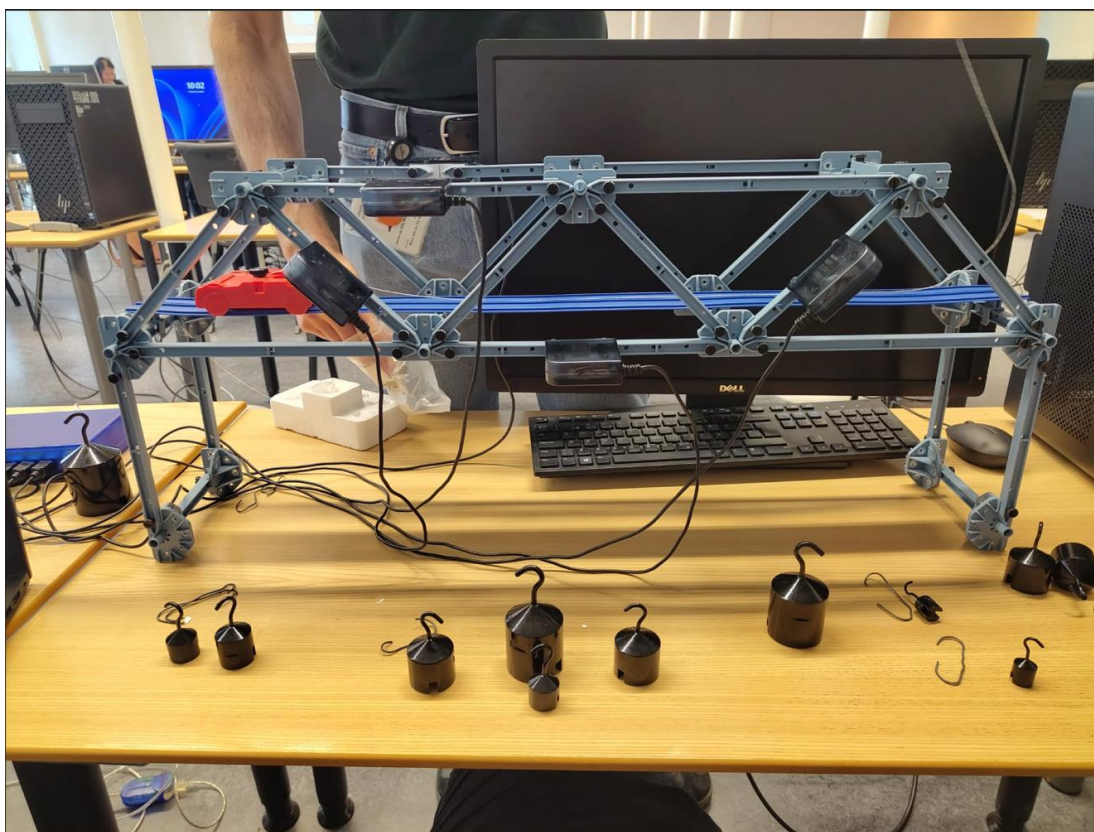
Bilde av broen under test 2

Test 3

E	A	B	D	C
Force 1 (N)	Force 2 (N)	Force 3 (N)	Force 4 (N)	Force 5 (N)
0,37N	-0,27N	0,37N	-0,04N	-0,77N



Måleresultat for test 3



Bilde av broen under test 3

DRØFTING OG FEILKILDER

- Det kan være belastninger vi ikke har tatt høyde for som påvirker måleresultatene
- Det kan være støy fra lastcellene til den elektriske leseren.

KONKLUSJON

Resultatene støtter hypotesen. Estimerte verdier er forholdsvis nærme kalkulerte verdier.

Vi lærte at et komplekst system om dette regnes ut som et likningssett, men også at vi ikke har hele helheten. Vi antar at dette er for

KILDER

- PDF-filen vi fikk til LAB1 i MAS149
- PDF-filen for Task2 i MAS149
- <https://ndla.no/r/biologi-1/rapportmal/0f3199dc47>
- <https://github.com/LB-Bachelor-mechanical-engineering/MAS149h2026/blob/main/obligatory/O2/workspace/andrei.ipynb>

Program

```
from sympy.core.relational import Equality, Relational, Unequality
import sympy as sp
from sympy import Eq, Matrix, Symbol, cos, rad, sin, symbols

sp.init_printing(use_unicode=True)

class Member2D:
    """Represents a 2D axial force member or external force with a
    scalar magnitude and a direction angle in degrees."""
    def __init__(self, name: str, angle_deg: float = 0, is_scalar:
bool = True):
        self.name = name
        self.angle_deg = angle_deg
        theta = rad(angle_deg)

        if is_scalar:
            # Single scalar magnitude (standard truss member force)
            self.magnitude = symbols(name)
            self.x = self.magnitude * cos(theta)
            self.y = self.magnitude * sin(theta)
        else:
            # Known vector component input
            self.x = symbols(f"{name}_x")
            self.y = symbols(f"{name}_y")

    def toMatrix(self):
        return Matrix([self.x, self.y])

    def __add__(self, other):
        if isinstance(other, Member2D):
            return VectorSum(self.toMatrix() + other.toMatrix())
        return VectorSum(self.toMatrix() + other)

    def __radd__(self, other):
        return self.__add__(other)

    def __sub__(self, other):
        if isinstance(other, Member2D):
            return VectorSum(self.toMatrix() - other.toMatrix())
        return VectorSum(self.toMatrix() - other)
```

```

def __neg__(self):
    return VectorSum(-self.toMatrix())

class VectorSum:
    """Helper wrapper to allow fluent addition/subtraction of 2D
    matrices."""

    def __init__(self, mat: Matrix):
        self.mat = mat

    def __add__(self, other):
        if isinstance(other, Member2D):
            return VectorSum(self.mat + other.toMatrix())
        return VectorSum(self.mat + getattr(other, "mat", other))

    def __sub__(self, other):
        if isinstance(other, Member2D):
            return VectorSum(self.mat - other.toMatrix())
        return VectorSum(self.mat - getattr(other, "mat", other))

    def toMatrix(self):
        return self.mat

# Define Member Forces (1 scalar unknown per member)
N_AC = Member2D("N_{AC}", angle_deg=0)
N_CD = Member2D("N_{CD}", angle_deg=0)
N_BD = Member2D("N_{BD}", angle_deg=0)
N_EF = Member2D("N_{EF}", angle_deg=0)
N_FG = Member2D("N_{FG}", angle_deg=0)
N_A = Member2D("N_{A}", angle_deg=90)
N_B = Member2D("N_{B}", angle_deg=90)

N_AE = Member2D("N_{AE}", angle_deg=45)
N_CE = Member2D("N_{CE}", angle_deg=135)
N_CF = Member2D("N_{CF}", angle_deg=45)
N_DF = Member2D("N_{DF}", angle_deg=135)
N_DG = Member2D("N_{DG}", angle_deg=45)
N_BG = Member2D("N_{BG}", angle_deg=135)

```

```

N_L1 = Member2D("N_{L1}", angle_deg=-90)
N_L2 = Member2D("N_{L2}", angle_deg=-90)
N_L3 = Member2D("N_{L3}", angle_deg=-90)
N_L4 = Member2D("N_{L4}", angle_deg=-90)

```

```

unknown_symbols = [
    symbols("N_{AC}"),
    symbols("N_{CD}"),
    symbols("N_{BD}"),
    symbols("N_{EF}"),
    symbols("N_{FG}"),
    symbols("N_{AE}"),
    symbols("N_{CE}"),
    symbols("N_{CF}"),
    symbols("N_{DF}"),
    symbols("N_{DG}"),
    symbols("N_{BG}"),
    symbols("N_{A}"),
    symbols("N_{B}"),
]

```

```

E: Equality | Relational | Unequality = Eq((N_EF-N_CE-
N_AE).toMatrix(), Matrix([0,0]))
F: Equality | Relational | Unequality = Eq((N_FG-N_EF-N_DF-
N_CF).toMatrix(), Matrix([0,0]))
G: Equality | Relational | Unequality = Eq((-N_FG-N_DG-
N_BG).toMatrix(), Matrix([0,0]))

```

```

# Known Loads (Downward vertical forces)
g:float = 9.81

```

```

L1:float=575e-3
L2:float=425e-3
L3:float=250e-3
L4:float=725e-3
Navn:str="Andrei"

```

```

# Substitute known load values
sub_loads = {
    symbols("N_{L1}"): L1 * g,

```

```

symbols("N_{L2}"): L2 * g,
symbols("N_{L3}"): L3 * g,
symbols("N_{L4}"): L4 * g,
}

# Joint Equilibrium Equations (Sum of forces = 0 at each node)
A: Equality | Relational | Unequality = Eq((-N_A+N_L1 + N_AC +
N_AE).toMatrix().subs(sub_loads), Matrix([0, 0]))
B: Equality | Relational | Unequality = Eq((-N_B+N_L4 + N_BG +
N_BD).toMatrix().subs(sub_loads), Matrix([0, 0]))
C: Equality | Relational | Unequality = Eq((N_L2 + N_CD + N_CE +
N_CF - N_AC).toMatrix().subs(sub_loads), Matrix([0, 0]))
D: Equality | Relational | Unequality = Eq((N_L3 + N_DF + N_DG -
N_CD - N_BD).toMatrix().subs(sub_loads), Matrix([0, 0]))

# move from vector to scalar functions
vector_eqs: list[Equality | Relational | Unequality] = [A, B, C, D,
E, F, G]
scalar_eqs: list[Equality | Relational | Unequality] = []

for eq in vector_eqs:
    scalar_eqs.append(Eq(eq.lhs[0], eq.rhs[0])) # Sum of Fx = 0
    scalar_eqs.append(Eq(eq.lhs[1], eq.rhs[1])) # Sum of Fy = 0

solution = sp.solve(scalar_eqs, unknown_symbols, dict=True)[0]

solutions: dict[str, float] = {}
for variable, value in solution.items():
    print(f"The force of {variable}\tis\t{value:+3.5f}[N]")
    solutions[variable] = value

print("\nNegative means rod 'N_{{a}}[b]] is in tension and is pulling
in")
print("positive means rod 'N_{{a}}[b]] is in compression and is
pushng out")
print(f"\nDette er for vektene til {Navn}, som er:")
print(f"\tL1: {L1*1e3:#.2f}\t[g]\n\tL2: {L2*1e3:#.2f}\t[g]\n\tL3:
{L3*1e3:#.2f}\t[g]\n\tL4: {L4*1e3:#.2f}\t[g]")

```

```

import numpy as np

c=np.cos(np.deg2rad(45))

A_x = solution[symbols("N_{AC}")] + solution[symbols("N_{AE}")] *
np.cos(np.deg2rad(45))
A_y = L1 * g + solution[symbols("N_{AE}")]*c

B_x = solution[symbols("N_{BD}")] + solution[symbols("N_{BD}")]*c
B_y = solution[symbols("N_{B}")] + solution[symbols("N_{BD}")]*c
Btot = ((B_x**2) + (B_y**2))**(1/2)
print(f"A_x:\t{A_x:+0.2f}")
print(f"A_y:\t{A_y:+0.2f}")
print(f"B_x:\t{B_x:+0.2f}")
print(f"B_y:\t{B_y:+0.2f}")
print(f"B: \t{Btot:+0.2f}")

```